Q-Learning Based Tic-Tac-Toe Game

Youtube:https://youtu.be/NVHrlGJrwjI

Github:Izzy-byte261/tic_tac_toe_qlearning

1.Introduction

This game is based on classic tic-tac-toe rules, featuring clear regulations, well-defined states and action spaces, and a rational reward function. It combines the Q-Learning algorithm from reinforcement learning to create a human-versus-AI match. The core objective is to train the AI agent through self-play, enabling it to master optimal tic-tac-toe strategies and ultimately provide human players with a challenging competitive experience.

2. Game Design

2.1 Rules of the Game

In this game, two players face off: a human controls the X side, while an AI controls the O side. Players take turns placing pieces on a 3×3 grid. The first player to connect their pieces in a continuous straight line (horizontal, vertical, or diagonal) wins. If the grid fills up without either player forming a line, the game ends in a draw. Basic rules are as follows: X-side moves first. Pieces may only land on empty spaces. The game ends immediately upon determining the winner or filling the board. The match consists of 10 games, with the final score deciding the overall winner. The AI opponent makes decisions and executes moves based on the real-time game situation.

2.2 State space and action space of Game

State space:The game state is represented by a 9-tuple, where each board cell has 3 possible states: 0 (empty), 1 (X, human player), 2 (O, AI player).

Action space:The action space is an integer ranging from 0 to 8, corresponding to the 9 positions on the board.

In the Q-Learning algorithm, the core lies in learning the value (Q-value) of "state-action pairs." Therefore, the keys in the Q-table are (state, action): where state is a tuple from the state space, and action is an integer from the action space.

The value in the Q-table represents the "expected cumulative reward" of taking that

action in that state. The AI selects the optimal action by optimizing the Q-values.

The state space describes “the current state of the board,” while the action space describes “what the player can do on the board.” Together, they form the core inputs for tic-tac-toe reinforcement learning.

2.3 UI design

The entire game interface follows a fresh pink-and-white color scheme and is built using Pygame. The game entry displays the title and subtitle, shows a decorative chessboard preview, lists the game rules, and allows players to click “START GAME” to begin the match. The game interface is as follows.

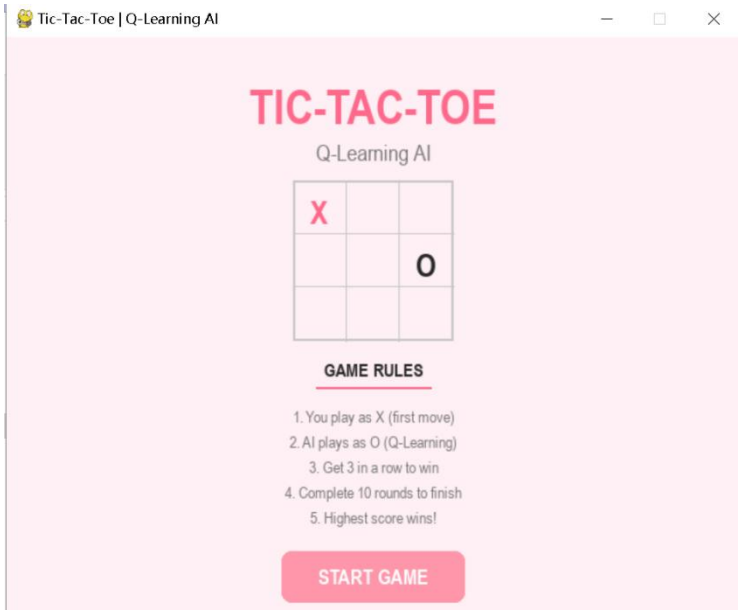

Fig 2- 1 Tic-Tac-Toe Game UI-1

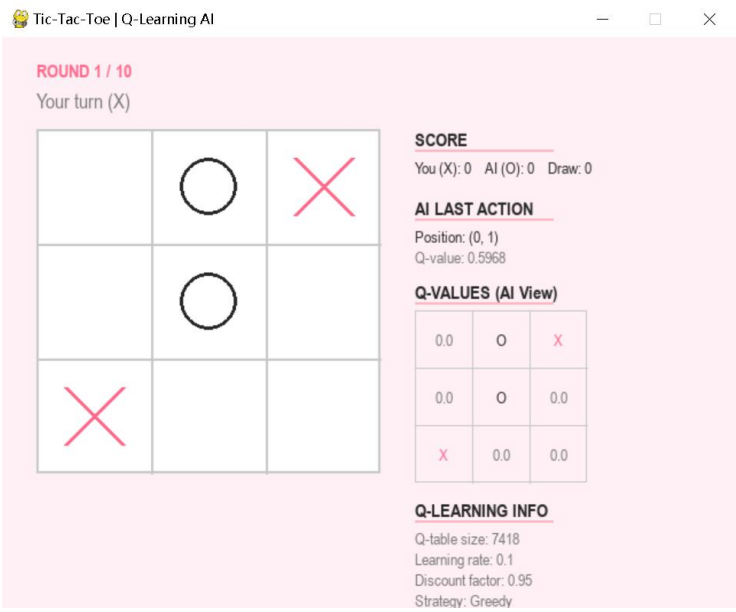

Fig 2- 2 In the game

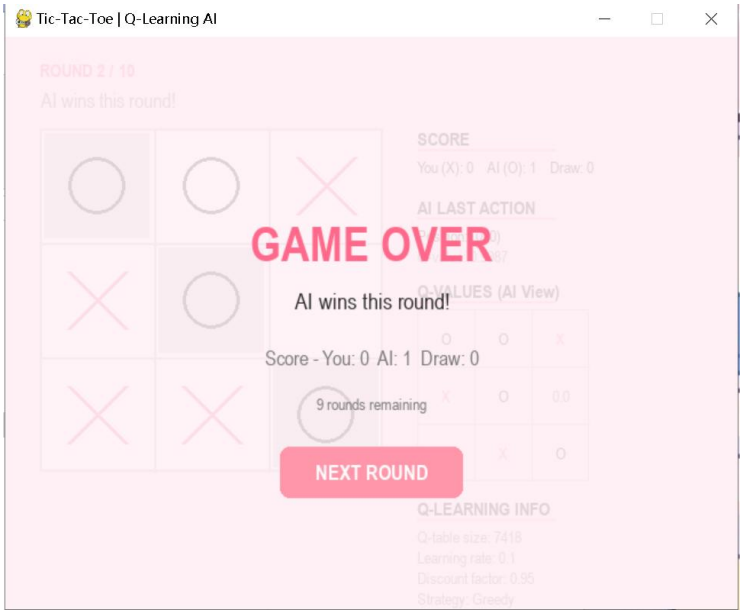

Fig 2- 3 Game over

2.4 Reward Function Design

The reward function is central to guiding the Q-Learning agent's learning. Its design follows the principle of rewarding positive behaviors, penalizing negative behaviors, and simultaneously encouraging efficient victories. The reward value is set within the game environment's step() method, and during training, the final reward is backpropagated to all historical actions. The specific reward rules are:

+1.0: Agent wins the game (achieves core objective);

-1.0: Agent loses the game (undesirable outcome);

+0.5: Game ends in a draw (neutral outcome, optimal result in a perfectly played tic-tac-toe match);

-0.1: For each valid move executed (penalizes redundant moves, encouraging the agent to win as quickly as possible).

3. Implementation of the Q-learning Algorithm

The project implements the standard Q-Learning algorithm within the Q-LearningAgent class. It designs a dual-agent self-play training mechanism (where X and O agents compete against each other) to learn optimal policies. This implementation fully leverages ε-greedy exploration, learning rate, and discount factor to balance exploration and exploitation. The trained agent policies maximize cumulative rewards.

3.1 Q-table Data Structure

```
self.q_table = {}  # {(state, action)
```

state = Board state, represented as a tuple of length 9, for example, (1, 0, 2, 0, 1, 0, 0, 0, 2)

action = Placement position 0-8

q_value = Expected reward from executing this action in this state, initialized to 0.0 by default.

3.2 Epsilon-Greedy Exploration Strategy

```
if training and random.random() < self.epsilon:
    return random.choice(valid_actions)
else:
    q_values = {a: self.get_q_value(state, a) for a in valid_actions}
    max_q = max(q_values.values())
    best_actions = [a for a, q in q_values.items() if q == max_q]
    return random.choice(best_actions)
```

Explore new strategies with probability epsilon.Utilize learned knowledge to select the currently known optimal action with probability 1 - epsilon.

Epsilon decays from 1.0 to 0.05, prioritizing extensive exploration early in training and shifting to exploitation later.

3.3 Q-value update formula

```
# Q(s,a) = Q(s,a) + α * [reward + γ * max Q(s',a') - Q(s,a)]
current_q = self.get_q_value(state, action)

if done:
    target = reward
else:
    max_next_q = max(self.get_q_value(next_state, a) for a in next_valid_actions)
    target = reward + self.gamma * max_next_q

new_q = current_q + self.lr * (target - current_q)
```

Q(s ,a)=Q(s ,a)+a*[reward + y * max Q(s' ,a') - Q(s , a)], Here, a denotes the learning rate, y represents the discount factor, reward is the immediate reward, s' is the next state, and max Q(s', a') is the maximum Q-value for the next state's valid action.

Parameters	Value	effect
a(learning rate)	0.1	Control the step size for each update. A larger step size results in faster learning but may lead to instability.
y(discount factor)	0.95	The weight of future rewards: the closer it is to 1, the greater the emphasis on long-term gains.
Epsilon decay	0.99995/step	Control the pace of transition from exploration to exploitation

3.4 Training Process

Two independent Q-Learning agents learn from each other through 100,000 self-play games. After each game concludes, the Q-values for all actions taken by both agents are updated based on the final outcome.

The draw rate rose from 15.1% to 66.0% — both sides learned to defend, making it difficult to defeat each other. This aligns with the theoretically optimal outcome of Tic-Tac-Toe (a perfect game must end in a draw).

X's first-move advantage gradually diminished — dropping from 56.3% to 23.8%, indicating that the O agent learned to counter the opening move.

The final Q-table size reached 8,052 entries — covering most reachable state-action pairs in the game.

Epsilon decay curve — from 1.0 (pure randomness) to 0.05 (95% utilization of

learned strategy), enabling thorough exploration early on and stable convergence later.

4. Challenges and Solutions

Tic-Tac-Toe only provides clear win/loss rewards at the end of the game, making it difficult to assess the quality of intermediate steps and resulting in sparse learning signals.

Solutions:

Fristly, Delayed Reward Backpropagation: After the game ends, update the Q-values for all intermediate steps based on the final outcome.

Second, Small step penalty: Introduce a -0.1 step penalty to encourage agents to win in the fewest moves.

Finally, Draw bonus: Award a +0.5 bonus for draws, teaching agents to at least avoid losing when victory is unattainable.

5. Summary

This project successfully implemented a Q-Learning-based Tic-Tac-Toe game:

At the algorithmic level: The complete Q-Learning algorithm was implemented, encompassing the Q-table, ε-greedy exploration, and Q-value updates. Through 100,000 self-play training matches, the agent learned a near-optimal strategy, achieving a draw rate of 66%—aligning with the theoretical optimal solution for Tic-Tac-Toe.

At the game interface level: A minimalist black-and-white graphical interface was built using Pygame, implementing the full game flow (Welcome → Match → Game Over → Match End). It supports a 10-match series format, displays scores and Q-values in real-time, and allows saving/loading Q-table data, eliminating the need for retraining after each session.

User experience: Features a clean pink-and-white UI style. The entry page displays game rules, with the right panel showing real-time Q-values for AI decisions. The final screen presents a complete 10-game match history and statistical results.

6. Reference

[1] Sutton R S, Barto A G. Reinforcement Learning: An Introduction[M]. MIT Press, 2018.

[2] Pygame Official Documentation. https://www.pygame.org/docs/

[3] KI-cheng. QlearningTankWar[EB/OL]. GitHub, 2025.

[4] Te. Tetris-deep-Q-learning-pytorch[EB/OL]. GitCode, 2025.